

Playwright-Notes-6

- **Page Object Model (POM) with fixtures-** POM is a design pattern in test automation that creates an object repository for storing all locators-POM can be implemented with fixtures, through that we would be minimising multiple page import and object creation in each test file for multiple pages

In VSCode:

1. Create a file called **POMFixtures.ts** under the **fixtures** folder (this is an extension of the files we created under the pages folder previously)
2. Include the below code in this file:

```
import {test as baseTest} from "@playwright/test";
import { LoginPage } from "../pages/LoginPage";
import { HomePage } from "../pages/HomePage";
import { CartPage } from "../pages/CartPage";

type MyPomFixtures = {
  loginPage : LoginPage;
  homePage : HomePage;
  cartPage : CartPage;
}

export const test = baseTest.extend<MyPomFixtures>({
  loginPage : async({page}, use)=>{
    const loginPage = new LoginPage(page);

    await use(loginPage);
  },

  homePage : async({context}, use)=>{
    const homePage = new HomePage(await context.newPage());
    await use(homePage);
  },

  cartPage : async({page}, use)=>{
    await use(new CartPage(page));
  },
});
```

```
})
```

Explanation of code:

This code combines two powerful patterns: Custom Fixtures + Page Object Model (POM). Let me break it all down:

The imports:

```
import { test as baseTest } from "@playwright/test"
import { LoginPage } from "../pages/LoginPage"
import { HomePage } from "../pages/HomePage"
import { CartPage } from "../pages/CartPage"
```

- **baseTest** — Playwright's original test to extend
 - **LoginPage, HomePage, CartPage** — your Page Object classes (each one wraps a page and contains its methods)
-

The type definition:

```
type MyPomFixtures = {
  loginPage: LoginPage
  homePage: HomePage
  cartPage: CartPage
}
```

Tells TypeScript: *"these 3 fixtures exist and here's what type each one returns"* — gives you autocomplete in tests.

loginPage fixture — uses **page**:

```
loginPage: async ({ page }, use) => {
  const loginPage = new LoginPage(page) // wraps the default page
  await use(loginPage)                  // hands the LoginPage object to the test
}
```

```
}
```

- Takes Playwright's built-in **page** fixture
 - Creates a **LoginPage** instance with it
 - No setup or teardown — just creates and hands it over
-

homePage fixture — uses **context**:

```
homePage: async ({ context }, use) => {  
  const homePage = new HomePage(await context.newPage()) // creates a  
  BRAND NEW tab  
  await use(homePage)  
}
```

- Instead of using the default **page**, it creates a new tab via **context.newPage()**
 - Useful when you need a separate tab for the home page (e.g. login happens on one tab, home on another)
-

cartPage fixture — shorthand style:

```
cartPage: async ({ page }, use) => {  
  await use(new CartPage(page)) // create and pass in one line  
}
```

- Same as **loginPage** but written more concisely
 - Creates **CartPage** inline without a variable
-

How a test uses this:

```
import { test } from "../fixtures/MyPomFixtures"
```

```
test("Add item to cart", async ({ loginPage, homePage, cartPage }) => {  
  await loginPage.goto()  
  await loginPage.login("user@test.com", "password")  
})
```

```

    await homePage.searchItem("shoes")
    await homePage.addToCart("Nike Air Max")

    await cartPage.verifyItem("Nike Air Max")
  })

```

Clean, readable — no `new LoginPage(page)` clutter inside tests.

The big picture:

baseTest

```

├── extended with POM fixtures
│   ├── loginPage → new LoginPage(page)    → same tab
│   ├── homePage  → new HomePage(new tab)   → new tab
│   └── cartPage  → new CartPage(page)      → same tab

```

Why this pattern is powerful:

Without POM Fixtures	With POM Fixtures
<code>new LoginPage(page)</code> in every test	Just ask for <code>loginPage</code>
Page setup logic scattered everywhere	All in one fixtures file
Hard to maintain	Change once, applies everywhere
Messy test files	Clean, readable tests

So this file is essentially a single source of truth for how your page objects get created and handed to tests.

- Now, create a file called **CartVerification_with_POM_and_fixtures.spec.ts** under the **tests** folder (this is a cleaner version of the `CartVerification_with_POM` we created previously)
- Include the below code:

```

import { expect } from "@playwright/test"
import { test } from "../fixtures/POMFixtures"

```

```

test("Verification of Cart", async ({ page, loginPage, homePage,
cartPage }) => {
    await loginPage.openApplication();
    await loginPage.login("standard_user", "secret_sauce");
    await expect(homePage.homePageHeading).toHaveText("Swag
Labs");
    await homePage.backPackAddToCart();
    await expect(homePage.cartIcon).toHaveText("1");
    await
expect(homePage.backpackRemoveButton).toBeVisible();
    await homePage.gotoCart();
    await expect(cartPage.backpackItemLink).toHaveText("Sauce
Labs Backpack");
})

```

Explanation of code:

This is the actual test file that brings everything together — imports, fixtures, page objects, and assertions all in one test.

The imports:

```

import { expect } from "@playwright/test" // for assertions
import { test } from "../fixtures/POMFixtures" // YOUR custom test with
POM fixtures

```

- **expect** comes from Playwright directly (no need to customize it)
- **test** comes from your fixtures file — gives access to **loginPage**, **homePage**, **cartPage**

The test signature:

```

test("Verification of Cart", async ({ page, loginPage, homePage,
cartPage }) => {

```

- **"Verification of Cart"** — the test name (shows in reports)

- **page** — Playwright's built-in page
 - **loginPage, homePage, cartPage** — your custom POM fixtures, auto-created before the test runs
-

Step by step walkthrough:

```
await loginPage.openApplication()
```

Opens the app URL — this method lives inside **LoginPage** class

```
await loginPage.login("standard_user", "secret_sauce")
```

Fills in username + password and submits — all that logic is hidden inside **LoginPage**

```
await expect(homePage.homePageHeading).toHaveText("Swag Labs")
```

After login, verifies you landed on the right page by checking the heading text

```
await homePage.backPackAddToCart()
```

Clicks "Add to Cart" for the backpack — method lives in **HomePage**

```
await expect(homePage.cartIcon).toHaveText("1")
```

Verifies the cart badge shows **1** after adding the item

```
await expect(homePage.backpackRemoveButton).toBeVisible()
```

Verifies the "Remove" button appeared — confirms item was added successfully

```
await homePage.gotoCart()
```

Clicks the cart icon to navigate to the cart page

```
await expect(cartPage.backpackItemLink).toHaveText("Sauce Labs Backpack")
```

Verifies the correct item appears in the cart

What each file is responsible for:

POMFixtures.ts → creates loginPage, homePage, cartPage objects

- ├── LoginPage.ts → openApplication(), login()
- ├── HomePage.ts → homePageHeading, backPackAddToCart(), cartIcon,
- └── CartPage.ts → backpackRemoveButton, gotoCart(), backpackItemLink

The full flow visualized:

Open app

▼

Login with credentials

▼

Verify "Swag Labs" heading ✓

▼

Add backpack to cart

▼

Verify cart badge shows "1" ✓

Verify Remove button visible ✓

▼

Go to Cart page

▼

Verify "Sauce Labs Backpack" in cart ✓

Why this test is clean and readable:

What you DON'T see	Where it actually lives
<code>page.goto('https://...')</code>	<code>LoginPage.openApplication()</code>
<code>page.fill('#user', ...)</code>	<code>LoginPage.login()</code>
<code>page.click('.add-to-cart')</code>	<code>HomePage.backPackAddToCart()</code>
Selectors like <code>#cart-count</code>	<code>HomePage.cartIcon</code>

The test reads almost like plain English — *open app* → *login* → *add to cart* → *verify cart* — because all the messy browser details are hidden inside the page object classes.

- **Fixtures vs BeforeEach/AfterEach hooks-**
- **beforeEach** hook is executed before every test
- **afterEach** hook is executed after each test

Referring to the below code in the `CartVerification_with_hooks.ts` file

```
import { test, expect } from "@playwright/test"

test.beforeEach(async ({ page }) => {
  //Login
  await page.goto("https://www.saucedemo.com/");
  //Login
  await page.locator('[data-test="username"]').fill("standard_user");
  //Login
  await page.locator('[data-test="password"]').fill("secret_sauce");
  await page.locator('[data-test="login-button"]').click();
})

test.afterEach(async ({page})=>{
  //Logout
  //Logout
})
```



```

        await page.getByRole('button', { name: 'Open Menu'
    }).click();
        await page.getByRole('link', { name: 'Logout' }).click();
    })

test("Adding / Removing item to / from Cart verification", async
({page}) => {
    //Adding and Removing Item from cart
    await page.getByText("Sauce Labs Backpack").click();

    await
page.locator('[data-test="add-to-cart-sauce-labs-backpack"]').cli
ck();
    await page.locator(".shopping_cart_link").click();
    await expect(page.getByRole('link', { name: 'Sauce Labs
Backpack' })).toHaveText("Sauce Labs Backpack")

    await
expect(page.locator('[data-test="remove-sauce-labs-backpack"]')).
toBeVisible();

    await
page.locator('[data-test="remove-sauce-labs-backpack"]').click();
    await expect(page.getByRole('link', { name: 'Sauce Labs
Backpack' })).not.toBeVisible()

})

test("Empty Cart verification", async ({page}) => {
    //Cheking Empty Cart
    await page.locator(".shopping_cart_link").click();

    await
expect(page.locator('.inventory_item_name')).not.toBeVisible();
})

```

Explanation of code:

This is a complete test file with shared setup/teardown and 2 tests for a shopping cart. Let me break it all down:

The imports:

```
import { test, expect } from "@playwright/test"
```

- Using Playwright's default **test** and **expect** — no custom fixtures needed here
 - All interactions are done directly with **page**
-

test.beforeEach — runs before EVERY test:

```
test.beforeEach(async ({ page }) => {  
  await page.goto("https://www.saucedemo.com/")  
  await page.locator('[data-test="username"]').fill("standard_user")  
  await page.locator('[data-test="password"]').fill("secret_sauce")  
  await page.locator('[data-test="login-button"]').click()  
})
```

- Opens the app
 - Fills in username and password
 - Clicks login
 - Every test starts already logged in — no need to repeat this in each test
-

test.afterEach — runs after EVERY test:

```
test.afterEach(async ({ page }) => {  
  await page.getByRole('button', { name: 'Open Menu' }).click()  
  await page.getByRole('link', { name: 'Logout' }).click()  
})
```

- Opens the hamburger menu
 - Clicks logout
 - Every test ends cleanly logged out — ensures no session bleeds into the next test
-

Test 1 — Adding and Removing from Cart:

```
test("Adding / Removing item to / from Cart verification", async ({ page }) => {  
  
  await page.getByText("Sauce Labs Backpack").click()  
  // clicks on the backpack product
```

```

                                                                    await
page.locator('[data-test="add-to-cart-sauce-labs-backpack"]').click()
// clicks "Add to Cart" button

await page.locator(".shopping_cart_link").click()
// navigates to the cart

    await expect(page.getByRole('link', { name: 'Sauce Labs Backpack'
}))toHaveText("Sauce Labs Backpack")
// ✅ verifies backpack appears in cart

                                                                    await
expect(page.locator('[data-test="remove-sauce-labs-backpack"]')).toBeV
isible()
// ✅ verifies Remove button is visible

await page.locator('[data-test="remove-sauce-labs-backpack"]').click()
// clicks Remove

    await expect(page.getByRole('link', { name: 'Sauce Labs Backpack'
})).not.toBeVisible()
// ✅ verifies backpack is GONE from cart
})

```

Test 2 — Empty Cart verification:

```

test("Empty Cart verification", async ({ page }) => {

    await page.locator(".shopping_cart_link").click()
    // goes straight to cart without adding anything

    await expect(page.locator('.inventory_item_name')).not.toBeVisible()
    // ✅ verifies cart is empty
})

```

The full execution flow for both tests:

Test 1:

```

beforeEach → open app → login
    |
    ▼

```

click backpack → add to cart → go to cart
verify backpack in cart ✓
verify remove button visible ✓
click remove
verify backpack gone ✓

↓
afterEach → open menu → logout

Test 2:

beforeEach → open app → login

↓
go to cart (empty)
verify no items in cart ✓

↓
afterEach → open menu → logout

beforeEach vs fixtures — what's the difference?

	beforeEach	Fixtures
Where defined	Same file	Separate fixtures file
Reusable across files	✗ No	✓ Yes
Good for	Quick, file-specific setup	Shared setup across many files
Returns a value to test	✗ No	✓ Yes

Key Playwright methods used:

Method	What it does
<code>page.goto(url)</code>	Navigates to a URL

<code>page.locator(selector)</code>	Finds an element by CSS/data-test
<code>page.getByText(text)</code>	Finds element by visible text
<code>page.getByRole(role)</code>	Finds element by ARIA role
<code>.fill(value)</code>	Types into an input field
<code>.click()</code>	Clicks an element
<code>expect().toHaveText()</code>	Asserts element has specific text
<code>expect().toBeVisible()</code>	Asserts element is visible
<code>expect().not.toBeVisible()</code>	Asserts element is NOT visible

In VSCode:

1. Create a file called **HooksFixture.ts** under the **fixtures** folder
2. Include the following code:

```
import { test as baseTest, type Page } from
"@playwright/test";

type MyHooksFixtures = {
  loginlogoutfixture: any;
}

export const test = baseTest.extend<MyHooksFixtures>({
  loginlogoutfixture: async ({ page }: { page: Page },
use: (value: any) => Promise<void>) => {
    const loginlogoutfixture = undefined;
    //Login
    await page.goto("https://www.saucedemo.com/");
```

```

                                                                    await
page.locator('[data-test="username"]').fill("standard_user")
;
                                                                    await
page.locator('[data-test="password"]').fill("secret_sauce");
                                                                    await
page.locator('[data-test="login-button"]').click();

    await use(loginlogoutfixture);

    //Logout
    await page.getByRole('button', { name: 'Open Menu'
}).click();
    await page.getByRole('link', { name: 'Logout'
}).click();

    }
})

export {expect} from "@playwright/test"

```

Explanation:

This code converts the **beforeEach/afterEach** hooks from the previous file into a reusable fixture. Let me break it down:

The imports:

```
import { test as baseTest, type Page } from "@playwright/test"
```

- **baseTest** — Playwright's original test to extend
- **type Page** — TypeScript type for the Playwright page object (used for type safety)

The type definition:

```
type MyHooksFixtures = {
  loginlogoutfixture: any
}
```

```
}
```

- Defines the shape of the fixture
 - **any** because this fixture doesn't return a useful value — its job is just to run login/logout, not pass data to the test
-

The fixture itself:

```
loginlogoutfixture: async ({ page }: { page: Page }, use) => {

  const loginlogoutfixture = undefined
  // undefined because we don't need to pass any value to the test
  // the fixture's only job is setup and teardown

  // ---- SETUP (beforeEach equivalent) ----
  await page.goto("https://www.saucedemo.com/")

  page.locator('[data-test="username"]').fill("standard_user")
  await page.locator('[data-test="password"]').fill("secret_sauce")
  await page.locator('[data-test="login-button"]').click()

  await use(loginlogoutfixture) // test runs here

  // ---- TEARDOWN (afterEach equivalent) ----
  await page.getByRole('button', { name: 'Open Menu' }).click()
  await page.getByRole('link', { name: 'Logout' }).click()
}
```

The last line:

```
export { expect } from "@playwright/test"
```

- Re-exports Playwright's **expect** from THIS file
- So your test files only need one import instead of two:

```
//  With re-export — one import
import { test, expect } from "../fixtures/MyHooksFixture"
```

```
//  Without re-export — two imports needed
```

```
import { test } from "../fixtures/MyHooksFixture"
import { expect } from "@playwright/test"
```

How a test uses this:

```
import { test, expect } from "../fixtures/MyHooksFixture"

test("Empty Cart verification", async ({ page, loginlogoutfixture })
=> {
  // already logged in ✓
  await page.locator(".shopping_cart_link").click()

  expect(page.locator('.inventory_item_name')).not.toBeVisible()
  // automatically logs out after ✓
})
```

Comparing the two approaches:

Previous approach (beforeEach/afterEach): (fixture):	This approach
<pre>test.beforeEach(async({page})=>{ async({page}, use)=>{ // login }) test("...", async({page})=>{ // test }) test.afterEach(async({page})=>{ // logout })</pre>	<pre>loginlogoutfixture: await use() ← test runs here // logout }</pre>

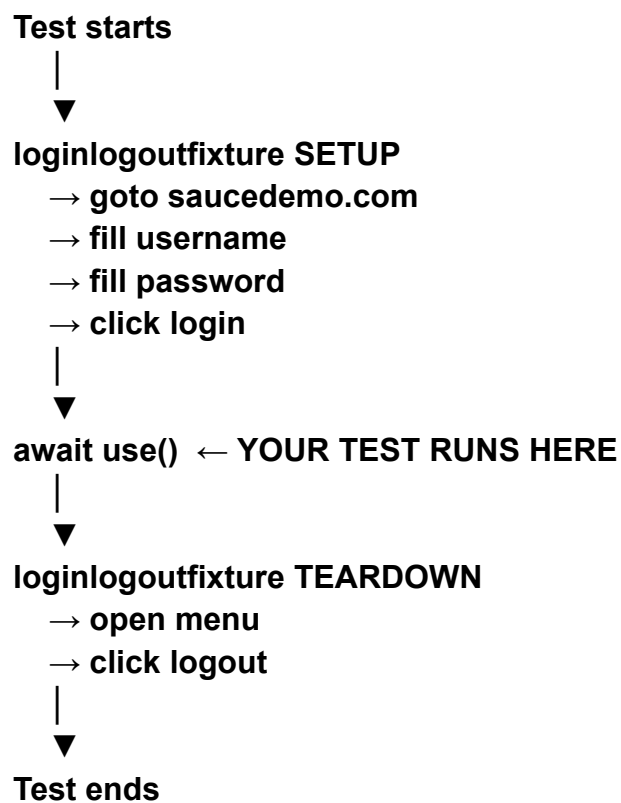
Why fixture is better than beforeEach/afterEach:

beforeEach

Fixture

	/afterEach	
Reusable across files	✗ No	✓ Yes
Setup + teardown together	✗ Split in two blocks	✓ One place
Opt-in per test	✗ Runs for ALL tests	✓ Only when requested
Re-export expect	✗ Not possible	✓ Yes

The full execution flow:



So this fixture is essentially `beforeEach` + `afterEach` bundled into one reusable, portable unit.

3. Create a test file called **Fixtures_over_hooks.spec.ts** under the **tests** folder and include the below code:

```
import {test, expect} from "../fixtures/HooksFixtures";

test("Adding / Removing item to / from Cart verification",
  async ({page, loginlogoutfixture}) => {
    //Adding and Removing Item from cart
    await page.getByText("Sauce Labs Backpack").click();
    await
page.locator('[data-test="add-to-cart-sauce-labs-backpack"]'
).click();
    await page.locator(".shopping_cart_link").click();
    await expect(page.getByRole('link', { name: 'Sauce Labs
Backpack' })).toHaveText("Sauce Labs Backpack")
    await
expect(page.locator('[data-test="remove-sauce-labs-backpack"
]')).toBeVisible();
    await
page.locator('[data-test="remove-sauce-labs-backpack"]').cli
ck();
    await expect(page.getByRole('link', { name: 'Sauce Labs
Backpack' })).not.toBeVisible()
  })

test("Empty Cart verification", async ({page,
loginlogoutfixture}) => {
  //Cheking Empty Cart
  await page.locator(".shopping_cart_link").click();
  await
expect(page.locator('.inventory_item_name')).not.toBeVisible
();
})

test("Add", async()=>{
  const a = 20;
  const b = 30;
```

```
const c = a + b;
console.log(c);
})
```

Explanation of code:

This is the test file that uses the **loginlogoutfixture** you built. Let me break it down:

The import:

```
import { test, expect } from "../fixtures/HooksFixtures"
```

- Single import gives you both **test** and **expect**
- This works because you re-exported **expect** from the fixtures file
- Both **loginlogoutfixture** and **page** are now available

Test 1 — Adding/Removing from Cart:

```
test("Adding / Removing item to / from Cart verification", async ({
  page, loginlogoutfixture }) => {
```

```
    await page.getByText("Sauce Labs Backpack").click()
    // clicks the backpack product
```

```
    // adds it to cart
    await page.locator("[data-test='add-to-cart-sauce-labs-backpack']").click()
    ()
```

```
    await page.locator(".shopping_cart_link").click()
    // navigates to cart
```

```
    await expect(page.getByRole('link', { name: 'Sauce Labs Backpack' })).toHaveText("Sauce Labs Backpack")
    // ✅ verifies backpack is in cart
```

```

                                                                    await
expect(page.locator('[data-test="remove-sauce-labs-backpack"]')).
toBeVisible()
    // ✅ verifies Remove button is visible

                                                                    await
page.locator('[data-test="remove-sauce-labs-backpack"]').click()
    // removes item from cart

    await expect(page.getByRole('link', { name: 'Sauce Labs
Backpack' })).not.toBeVisible()
    // ✅ verifies backpack is gone from cart
})

```

Test 2 — Empty Cart:

```

test("Empty Cart verification", async ({ page, loginlogoutfixture })
=> {

    await page.locator(".shopping_cart_link").click()
    // goes straight to cart without adding anything

                                                                    await
    expect(page.locator('.inventory_item_name')).not.toBeVisible()
    // ✅ verifies cart is empty
})

```

Test 3 — Add (simple math, no browser):

```

test("Add", async () => {
    const a = 20
    const b = 30
    const c = a + b
    console.log(c) // prints 50
})

```

- No **page**, no **loginlogoutfixture** — just pure logic
- No browser opens for this test
- Notice **loginlogoutfixture** is NOT requested — so login/logout never runs for this test

This is the key behavior to understand:

Test 1: { page, loginlogoutfixture }

- fixture setup (login) ✓
- test runs
- fixture teardown (logout) ✓

Test 2: { page, loginlogoutfixture }

- fixture setup (login) ✓
- test runs
- fixture teardown (logout) ✓

Test 3: {} ← nothing requested

- fixture setup SKIPPED ✓
- test runs
- fixture teardown SKIPPED ✓

Playwright only runs fixture setup/teardown if the test asks for it.

Full execution flow:

► **Test 1**

loginlogoutfixture **SETUP** → login
click backpack
add to cart
go to cart
verify backpack in cart ✓
verify remove button ✓
click remove
verify backpack gone ✓
loginlogoutfixture **TEARDOWN** → logout

► **Test 2**

loginlogoutfixture **SETUP** → login
go to cart
verify cart empty ✓
loginlogoutfixture **TEARDOWN** → logout

► **Test 3**

(no fixture)

```
a + b = 50
console.log(50)
```

Summary table:

Uses

page

Uses

loginlo

goutfix

ture

Browser

opens

Login

runs

Logout

runs

Test 3 proves that fixtures are opt-in — if you don't ask for them, they simply don't run.

Advantages of Fixtures over BeforeEach/AfterEach hooks-

- Fixtures encapsulate setup and teardown in the same place so its easier to write
- Fixtures are reusable between test files-you can define them once in all your tests-thats how Playwright's built-in fixtures work
- Fixtures are on-demand-you can define as many fixtures as you'd like and Playwright will setup only the ones needed by your test and nothing else
- Fixtures are composable-they can depend on each other to provide complex behaviours
- Fixtures are flexible-Tests can use any combination of fixtures to tailor the precise environment they need, without affecting other tests

- Fixtures simplify grouping-you are free to wrap tests by their meaning and no longer need to wrap tests in describes block which sets up the environment